

03

1시간

핵심 기능 탐색

슬래시 커맨드

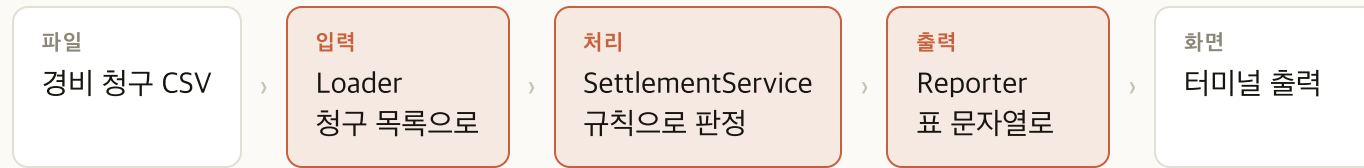
도구 호출

권한 설정

컨텍스트 관리

첫 규칙

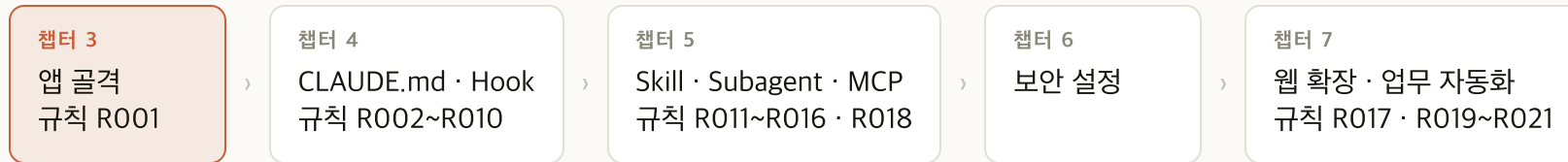
실습 개요: 경비 정산 시스템



- **단방향 전달:** 각 단계는 앞 단계의 결과만 받습니다. 처리 단계는 파일을 모르고, 출력 단계는 규칙을 모릅니다.
- **교체 가능한 양 끝:** 입력과 출력 단계는 바꿔 끼울 수 있고, 가운데 처리 단계는 그대로 남습니다.

사내 경비 규정 20개에 맞춰 경비 청구를 검사하는 프로그램입니다.
규정 한 항목이 규칙 파일 하나가 되고, 이 챕터에서 그중 첫 번째를 만듭니다.

실습 전체 구성



- **저장소 하나로 진행:** 챕터 2에서 클론한 `C:\Users\Administrator\lab` 을 마지막 챕터까지 이어서 씁니다. 챕터마다 새로 시작하지 않습니다.
- **함께 쌓이는 두 가지:** 하나는 경비 정산 앱의 코드, 다른 하나는 그 코드를 다루는 방식을 적어 두는 `.claude/` 하네스입니다.

각 챕터의 작업은 앞 챕터에서 만든 것 위에 쌓입니다.
규칙은 챕터마다 늘어나고, 그 규칙을 다루는 방식은 하네스에 남습니다.

이 챕터에서 다루는 내용

- **관찰 중심 진행:** 규칙 하나를 실제로 만들면서 도구 호출, 권한, 컨텍스트가 어떻게 움직이는지 화면에서 확인합니다.
- **함께 생기는 앱 골격:** 첫 규칙을 만들면 로더와 서비스, 컨테이너, CLI가 최소 형태로 갖춰집니다. 이후 챕터에서는 규칙만 늘어납니다.
- **CLAUDE.md 없는 첫 구현:** 이 챕터에서는 CLAUDE.md 를 만들지 않습니다. 같은 설명을 반복해 보아야 이 파일이 왜 필요한지 알게 됩니다.

이 챕터가 끝나면 `uv run pytest` 에서 `1 passed` 가 나옵니다. 진도는 통과한 테스트 건수로 확인합니다.

입력창의 접두사

접두사	하는 일
/	커맨드나 Skill 실행
!	셸 명령을 그대로 실행하고 출력을 세션에 넣음
@	파일 경로 자동완성

셸 명령을 실행하려고 터미널을 따로 열 필요가 없습니다.

앞에 `!` 를 붙이면 Claude Code 안에서 그대로 실행됩니다. `!git status` 처럼 씁니다.

주요 슬래시 커맨드

커맨드	하는 일
/help	커맨드 목록 표시
/context	컨텍스트 점유 내역 표시
/clear	대화 비우기: 앞의 내용은 사라짐
/compact	대화 요약: 흐름은 유지
/model	모델 선택
/usage	이번 세션의 토큰 사용량과 비용 표시

여섯 개만 먼저 익히세요.

입력창에 **/** 를 입력하면 목록이 나타납니다. 다른 주요 커맨드는 이후 챕터에서 다룹니다.

컨텍스트 창의 점유 내역

/context

Context Usage 48.5k/967k (5%)

Estimated usage by category

System prompt	9.4k	(1.0%)
System tools	29.4k	(3.0%)
Skills	2.8k	(0.3%)
Messages	7k	(0.7%)
Free space	885.5k	(91.6%)
Autocompact buffer	33k	(3.4%)

- **가장 큰 항목:** 도구 정의입니다. 대화를 시작하기 전부터 차 있습니다.
- **빠져 있는 Memory files:** CLAUDE.md 를 아직 만들지 않아 이 항목이 나타나지 않습니다. 챕터 4에서 생깁니다.
- **읽을수록 커지는 Messages:** 큰 파일을 통째로 읽게 하지 않는 이유입니다.

지금 /context 를 실행해 보세요.

화면에 표시되는 사용량은 설정 상태에 따라 사람마다 다릅니다.

/clear 와 /compact 의 차이

/clear

- 대화를 **통째로 비웁니다.** 앞에서 읽은 파일도 잊습니다.
- 주제가 완전히 바뀔 때 사용합니다.
- 규칙 하나를 끝내고 다음 규칙으로 넘어갈 때가 그런 경우입니다.

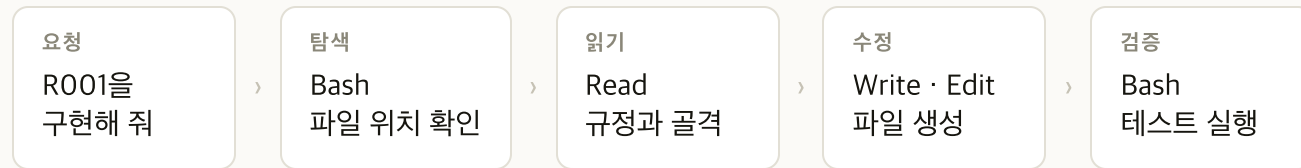
/compact

- 대화를 **요약해 줄입니다.** 지금까지의 흐름은 남습니다.
- 같은 작업을 이어 가야 하는데 공간이 부족할 때 사용합니다.
- 공간이 거의 차면 요청하지 않아도 자동으로 실행됩니다.

요약에는 손실이 따릅니다.

방금 정한 규칙이나 세부 조건은 요약 과정에서 빠질 수 있습니다. 중요한 결정은 대화에만 남기지 말고 파일에 기록하세요.

요청에서 도구 호출까지



- **좁혀서 읽는 방식:** 파일을 전부 열지 않고 검색으로 필요한 부분만 읽습니다.
- **스스로 하는 검증:** 테스트가 실패하면 결과를 보고 다시 고칩니다.

통과 조건은 요청에 함께 적어 주세요.

구현해 줘 만 적어도 검증까지 시도하지만, 무엇을 통과로 볼지는 Agent가 정합니다. 테스트를 만들고 통과할 때까지 고쳐 줘 처럼 조건까지 적어 주면 판정 기준이 매번 같아집니다.

도구 호출 화면 구성

진행 중 화면

```
● Read(docs/경비규정.md)
  ↳ Read 161 lines

● Write(rules/r001_required_fields.py)
  Do you want to create this file?
  > 1. Yes
    2. Yes, and don't ask again
    3. No, tell Claude what to do
```

- **호출 대상:** 괄호 안의 경로가 실제로 읽거나 쓰는 대상입니다. 의도한 파일이 아니면 `Esc` 로 중단하세요.
- **결과 요약:** 아래 들여쓰 줄에 읽은 줄 수나 검색 건수가 나옵니다.
- **승인 대상:** 프로젝트 안의 파일을 읽는 도구는 묻지 않고 실행되고, 파일을 바꾸는 도구와 셀 명령에만 승인 프롬프트가 나타납니다.

`Esc` 는 누른 횟수에 따라 되돌리는 범위가 다릅니다.

한 번 누르면 진행 중인 도구 호출이 중단되고, 두 번 누르면 편집 도구가 바꾼 파일이 직전 상태로 복구됩니다. 셀 명령으로 생긴 변경은 복구 대상이 아닙니다.

권한 모드 네 가지

	파일 편집	명령 실행	사용 상황
manual	매번 확인	매번 확인	기본값. 무엇을 묻는지 살펴볼 때
accept edits	자동 승인	매번 확인	편집이 반복될 때
plan	하지 않음	하지 않음	먼저 계획만 볼 때
auto	자동 승인	안전 검사 후 진행	닫겨 두고 결과를 볼 때

이 챕터는 manual 로 진행합니다.

어떤 도구가 승인을 요구하는지 확인하기 위해서입니다. 모드는 `Shift+Tab` 으로 바꿉니다.

승인 프롬프트의 선택지

- **Yes** : 이 호출만 허용합니다. 다음에 같은 도구가 호출되면 승인 프롬프트가 다시 나타납니다.
- **Yes, and don't ask again for ...** : 같은 형태의 호출을 허용하는 규칙을 `.claude/settings.local.json` 에 기록합니다. 다음 세션에도 남습니다.
- **No** : 이 호출을 거절합니다. 무엇을 다르게 할지 이어서 적어 주면 됩니다.

두 번째 선택지가 설정 파일을 만드는 유일한 자동 경로입니다.

나머지 규칙은 파일을 직접 열어 적습니다. 어떤 파일에 쓸지는 이어서 다룹니다.

설정 파일 세 곳

파일	적용 범위	저장소 커밋
<code>~/.claude/settings.json</code>	사용자의 모든 프로젝트	대상 아님
<code>.claude/settings.json</code>	이 프로젝트. 팀 전체가 공유	커밋함
<code>.claude/settings.local.json</code>	이 프로젝트. 개인 전용	커밋하지 않음

팀이 공유하는 규칙과 개인 설정을 섞지 마세요.

본인 계정에서만 유효한 경로나 임시 허용은 `settings.local.json` 에 둡니다. 이 구분은 챕터 6에서 API 키를 다룰 때 다시 나옵니다.

permissions 규칙 표기

.claude/settings.local.json · 자동 생성이 아니라 직접 작성하는 예시

```
{ "permissions": {
  "allow": ["Bash(uv run pytest:*)", "Read(docs/**)"],
  "deny":  ["Bash(git push:*)", "Edit(tests/golden/**)"]
} }
```

- 규칙은 **도구 이름**과 **대상**으로 씁니다. `Bash` 는 명령을, `Read` 와 `Edit` 는 경로를 받습니다. `deny` 가 `allow` 보다 우선이므로 둘 다 해당 하면 차단됩니다.
- **명령 규칙의 접미사 표기**: 명령 뒤에는 `:*` 를 붙입니다. `Bash(git status *)` 처럼 공백으로 적으면 인자 없이 실행한 `git status` 에 는 규칙이 적용되지 않습니다.

도구와 경로 단위로 Agent의 활동 범위를 정할 수 있습니다.

`deny` 에 디렉터리나 파일 경로를 적어 두면 수정 요청이 들어와도 그 경로는 바뀌지 않습니다.

실습 저장소의 구성

C:\Users\Administrator\lab

```
docs/          요구사항 · 경비규정 · 데이터 스키마
src/
  core/domain/  models.py · rules/base.py
  core/service/ input · usecase · output
  application/  container.py · cli/
tests/
  rules/fixtures/ r001.csv ~ r020.csv
  golden/base/    input.csv · expected.json
samples/         부서별 청구 파일 5개
```

- **선언만 있는 골격:** 클래스와 메서드 이름, 상속 관계만 정의돼 있습니다.
- **고정된 이름:** 테스트와 이후 예제가 이 이름을 참조하므로 바꾸지 않습니다.
- **아직 없는 규칙 파일:** `core/domain/rules/` 아래에 규칙마다 하나씩 만듭니다.

`claude` 를 실행한 폴더가 프로젝트 루트가 됩니다.

`cd C:\Users\Administrator\lab` 으로 이동한 뒤 `claude` 를 실행하세요. Claude Code는 파일과 설정을 이 경로 기준으로 찾습니다.

실습 명세 문서 세 개

파일	내용	참고 시점
요구사항.md	처리 흐름, 프로젝트 구조, 테스트가 기대하는 계약	무엇을 어디에 만들지 정할 때
데이터-스키마.md	청구 파일 형식, 도메인 모델 네 종류와 그 속성	모델을 채울 때
경비규정.md	검증할 규정 20개. 항목마다 조건과 반례 문구	규칙을 구현할 때

문서에 명시된 컬럼명을 그대로 데이터 모델의 속성명으로 쓰세요.

CSV 헤더가 `claim_id` 이면 `Expense` 의 속성도 `claim_id` 로 정의합니다. 둘 사이에 대응표를 두지 않습니다.

실습 1 · 도메인 모델 채우기

- 1 `C:\Users\Administrator\lab` 에서 `claude` 실행
- 2 문서 읽기 요청: `docs/데이터-스키마.md` 를 읽고 구조를 파악해 줘
- 3 모델 채우기 요청: 스키마 문서에 맞춰 `src/core/domain/models.py` 의 네 클래스를 채워 줘
- 4 승인 프롬프트 내용 확인 후 승인
- 5 `uv run python -c "from core.domain.models import Expense"` 로 импорт 확인

정확한 위임의 전제는 문서화입니다.

두 요청 어디에도 속성 이름과 타입이 없습니다. 그 내용이 문서에 이미 적혀 있어서 “스키마 문서에 맞춰” 한마디면 충분합니다. 요구사항을 문서로 정리해 둘수록 맡길 수 있는 일의 범위가 넓어집니다.

실습 2 · 권한 규칙 만들기

- 1 명령 실행 요청: `uv run pytest` 를 실행해 줘
- 2 프롬프트에서 `Yes, and don't ask again` 선택
- 3 생성 확인: `.claude/settings.local.json` 이 생기고 `allow` 에 규칙이 추가됐는지 확인
- 4 골든 데이터 잠금: `.claude/settings.local.json` 을 직접 열어 `deny` 에 `Edit(tests/golden/**)` 추가
- 5 `tests/golden` 아래에 파일을 만들어 달라고 요청해 차단 여부 확인

골든 데이터는 정답 판정의 근거이므로 사람도 Agent도 수정하지 않습니다.

이 차단 규칙은 구현이 틀렸을 때 정답을 고쳐 맞추는 일을 막습니다.

실습 3 · 입력 계층 채우기

- 1 `src/core/service/input/base.py` 를 열어 두 클래스의 역할 확인
- 2 공통 동작 요청: `FileLoader` 의 `load` 를 채워 줘
- 3 형식별 구현 요청: 스키마 문서에 맞춰 `CsvLoader` 를 채워 줘
- 4 파일을 여는 코드가 `CsvLoader` 에 중복되지 않았는지 확인

구현할 메서드	하는 일
<code>FileLoader.load</code>	파일을 읽어 <code>parse</code> 에 넘김
<code>CsvLoader.supports</code>	확장자가 <code>.csv</code> 인지 판별
<code>CsvLoader.parse</code>	헤더와 행을 해석해 <code>Expense</code> 생성

데이터-스키마.md 에는 인코딩과 그 근거가 함께 적혀 있습니다.

요청에 인코딩을 적지 않아도 `utf-8-sig` 로 열리는 것은 그 때문입니다. Excel에서 저장한 CSV라 `utf-8` 로 읽으면 첫 컬럼명이 깨진다는 설명까지 들어 있습니다.

R001 규정과 구현 방침

docs/경비규정.md

R001 · 필수 항목 누락 (단건)

```
claim_id employee_id department
used_date claim_date expense_type
amount payment_method merchant
```

중 하나라도 비어 있으면 반려합니다.

> 필수 항목이 비어 있습니다: {항목명}

- 규칙 하나에 파일 하나: `rules/r001_required_fields.py` 로 만듭니다.
- 규칙 이름 속성: 규정 제목을 `name` 에 그대로 씁니다.
- 한 건만 보는 판정: `RowRule` 을 상속합니다.
- 함께 구현할 판정 서비스: `RuleBasedSettlementService` 의 `settle` 도 이번에 채웁니다. 비어 있으면 규칙이 호출되지 않습니다.

Violation 의 속성은 `데이터-스키마.md` 에 정해져 있습니다.

규칙 코드와 이름도 속성으로 정의돼 있고, 값은 규칙이 채웁니다. 그래서 출력 계층은 이름을 따로 관리하지 않습니다.

조립과 등록 지점

- **조립:** 규칙과 로더, 리포터 같은 구현체를 골라 서비스에 끼우는 일입니다. 이 코드를 `container.py` 로 모으면 나머지 코드는 구현체를 몰라도 됩니다.
- **두 벌로 분리:** 조립 코드는 앱용과 테스트용을 따로 둡니다.
- **수동 등록:** 규칙 파일을 폴더에서 자동으로 찾지 않습니다. 규칙을 만들 때마다 아래 세 곳을 직접 수정합니다.

위치	작업
<code>rules/rXXX.py</code>	규칙 구현
<code>application/container.py</code>	앱 조립 코드에 등록
<code>tests/container.py</code>	테스트 조립 코드에 등록. 픽스처와 테스트도 작성

규칙이 20개로 늘면 같은 절차를 그만큼 되풀이해야 하고, 그때마다 같은 설명을 다시 적어야 합니다. 그 반복을 줄이는 것이 챕터 4와 챕터 5의 목표입니다.

실습 4 · 규칙 R001 구현

- 1 규칙 구현 요청: 경비규정 R001 을 구현해 줘. 파일은 `rules/r001_required_fields.py` 로 만들어 줘
- 2 만들어진 `rules/r001_required_fields.py` 를 열어 상속한 클래스가 `RowRule` 인지 확인
- 3 판정 서비스 요청: `RuleBasedSettlementService` 의 `settle` 을 채워 줘. 요구사항 문서대로 주입받은 규칙을 단건과 다건으로 나눠 적용해 줘

요청에 규정 문구를 옮겨 적지 않습니다.

규정 문서가 저장소에 있으므로 “경비규정 R001” 한마디로 판정 조건과 반려 문구가 정해집니다. 판정 결과가 규정과 다르면 요청에 조건을 덧붙이는 대신 규정 문서를 고칩니다.

실습 5 · 앱 실행 구성

- 1 **출력 계층 요청:** `ConsoleReporter` 의 `format` 을 채워 줘
- 2 **조립 요청:** `application/container.py` 의 세 함수를 채워 줘
- 3 **진입점 요청:** `cli/main.py` 의 `main` 을 채워 줘. 로더와 리포터, 서비스는 `container.py` 의 세 함수에서 받아 쓰고, 확장자로 입력 형식을 정하는 일은 `main` 안에서 처리해 줘
- 4 **실행:** `uv run settle samples\영업1팀.csv`

구현 대상	하는 일
ConsoleReporter	판정 결과를 표 문자열로 변환
build_loader	입력 형식에 맞는 로더 선택
build_service	규칙을 넣은 서비스 조립
build_reporter	출력 형식에 맞는 리포터 선택

진입점은 구현체를 직접 고르지 않습니다.

`main` 안에서 `CsvLoader` 를 바로 부르면 형식을 하나 더할 때마다 진입점을 고쳐야 합니다. 조립을 거치면 진입점은 형식 이름만 넘기고, 어떤 구현체를 쓸지는 `container.py` 한 곳에서 정해집니다.

실습 5 실행 결과

uv run settle samples\영업1팀.csv

청구ID	성명	사용일자	계정과목	금액	판정	위반
EXP-2026-0172	김민준	2026-05-01	접대비	180,000	통과	
EXP-2026-0173	이서연	2026-05-04	식비	26,000	통과	
EXP-2026-0221	장태현	2026-06-22	소모품비	31,000	반려	R001

전체 51건 · 통과 50 · 반려 1

R001 필수 항목 누락 1건

규칙을 추가할수록 이 표의 반려가 늘어납니다.

지금은 R001만 검사하므로 51건 가운데 1건만 반려됩니다. 요약에 보이는 규칙 이름은 Violation 에 담긴 값입니다.

테스트 조립 코드

tests/container.py · 규칙 등록과 서비스 조립

```
from core.domain.rules.r001_required_fields import R001
from core.service.usecase.settlement import RuleBasedSettlementService

def all_rules():          # 구현한 규칙을 여기에 등록
    return [R001()]

def build_service(rules=None):
    return RuleBasedSettlementService(
        all_rules() if rules is None else rules)
```

테스트 조립 코드는 앱 조립 코드와 따로 둡니다.

앱 조립은 로더와 리포터까지 끼우지만 테스트는 규칙만 있으면 됩니다. 테스트가 앱 조립에 의존하면 CLI나 웹이 바뀔 때 테스트도 함께 고쳐야 합니다.

테스트 픽스처 분리

tests/conftest.py · pathlib, pytest 임포트 생략

```
from container import build_service
from core.service.input.csv_loader import CsvLoader

FIXTURES = Path(__file__).parent / "rules" / "fixtures"
@pytest.fixture
def settle():
    def _settle(rules, fixture_name):
        expenses = CsvLoader().load(FIXTURES / fixture_name)
        return build_service(rules).settle(expenses)
    return _settle
```

픽스처는 테스트마다 반복되는 준비 과정을 모아 둔 코드입니다. `conftest.py` 에 두면 같은 폴더의 모든 테스트가 이 픽스처를 함께 씁니다.

실습 6 · 테스트 작성

- 1 **테스트 조립 요청:** `tests/container.py` 에 테스트용 조립 코드를 만들어 줘. 구현한 규칙을 등록하고, 규칙 목록을 인자로 받으면 등록된 규칙 대신 그 목록을 쓰게 해 줘
- 2 **픽스처 요청:** `tests/conftest.py` 에 픽스처를 만들어 줘. 규칙 목록과 픽스처 파일 이름을 받아 판정 결과를 돌려주게 해 줘
- 3 **테스트 요청:** `tests/rules/test_r001.py` 를 만들어 줘. `r001.csv` 네 행의 판정 결과를 기댓값과 대조하게 해 줘
- 4 `uv run pytest` 로 `1 passed` 확인

픽스처 데이터는 저장소에 이미 있습니다.

`tests/rules/fixtures/r001.csv` 에 위반 두 건과 통과 두 건이 들어 있습니다. 만드는 것은 그 파일을 읽어 판정과 기댓값을 대조하는 코드입니다.

실습 6 실행 결과

tests/rules/test_r001.py

```
from core.domain.rules.r001_required_fields import R001RequiredFields

def test_r001(settle):
    result = settle([R001()], "r001.csv")
    codes = [[v.code for v in j.violations]
              for j in result.judgements]
    assert codes == [["R001"], ["R001"], [], []]
```

uv run pytest

```
collected 1 item

tests/rules/test_r001.py .          [100%]

===== 1 passed in 0.03s =====
```

규칙 하나만 넣고 실행합니다.

픽스처에 다른 규칙 위반이 섞여도 이 테스트는 영향을 받지 않고, 실패하면 원인이 그 규칙 하나로 좁혀집니다.

세션 기록 파일에 남은 개인정보

- 1 **개인정보 검색 요청:** 이번 세션의 기록 파일(C:\Users\Administrator\.claude\projects\ 아래 .jsonl)에서 주민등록번호 패턴(\d{6}-\d{7}), 카드번호 패턴(\d{4}-\d{4}-\d{4}-\d{4}), 계좌번호로 보이는 패턴(은행명 + 숫자-숫자-숫자 형태)을 찾아서 찾은 값을 콘솔에 출력해 줘
- 2 화면에 출력된 값이 samples\영업1팀.csv 의 원본과 일치하는지 확인
- 3 그 값이 규칙 판정에 한 번도 쓰이지 않았음을 확인

찾은 값이 세션 기록에 남았다는 것은 이미 Claude API로 전송됐다는 뜻입니다.

세션 기록 파일은 모델과 주고받은 대화를 그대로 적어 둔 파일입니다. 도구가 돌려준 파일 내용도 대화의 일부이므로, 그 뒤 요청마다 같은 값이 다시 전송됩니다.

이 챕터에서는 차단이나 항목 제외를 적용하지 않습니다.

계좌번호와 주민등록번호는 지금 처리에 필요해 원본에 함께 들어 있습니다. 어떤 경로를 막고 어떤 항목을 가릴지는 챕터 6에서 판단 기준을 세운 뒤 설정합니다.

컨텍스트 정리 시점

지금까지 쌓인 내역

- 읽은 문서 세 개와 소스 파일 여러 개
- 실패한 테스트 출력과 다시 고친 이력
- 승인 프롬프트에서 주고받은 대화

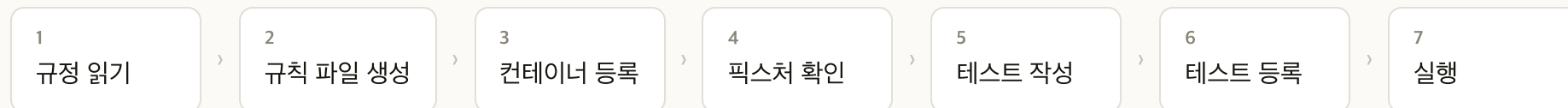
이 시점에 수행할 작업

- `/context` 로 처음과 비교
- 다음 규칙으로 넘어가기 전 `/clear` 실행
- 같은 규칙을 이어 갈 때 `/compact` 실행

`/clear` 를 실행하면 방금 정한 계층 규칙과 컨벤션도 함께 사라집니다.

다음 규칙에서 같은 설명을 처음부터 다시 하게 됩니다.

반복되는 절차 정리



- **규칙마다 거치는 일곱 단계:** 규칙 하나를 만들 때마다 이 순서를 그대로 따르며 같은 설명을 다시 적습니다.
- **고정된 전제:** 계층 규칙, 파일 위치, 등록 지점은 프로젝트가 이미 정해 둔 사항입니다.

챕터 4에서 이 설명을 `CLAUDE.md` 에 옮깁니다. 등록을 빠뜨리면 곧바로 잡아내는 장치도 Hook으로 만듭니다.

확인 항목

- ☐ `/context` 실행 후 사용량 확인
- ☐ `models.py` 네 클래스 속성 작성 완료
- ☐ 권한 모드 네 가지 차이 이해
- ☐ `deny` 규칙으로 골든 데이터 수정·생성 차단 확인
- ☐ `uv run pytest` 에서 `1 passed` 확인
- ☐ `/clear` 와 `/compact` 차이 이해
- ☐ `FileLoader` · `CsvLoader` 구현 완료
- ☐ `.claude/settings.local.json` 생성 확인
- ☐ 규칙 `R001` 과 `settle` 구현, 조립 코드 두 곳에 등록 완료
- ☐ `uv run settle` 실행 결과 표 확인